

What Your Transformer Is Forgetting (And Why That's Actually Fine)

Standard attention remembers everything perfectly — and that perfectionism is quietly breaking your GPU budget.

DataForge 2026 Submission — Memory vs. State

The filing cabinet that never stops growing

Imagine a student who writes down every word of every lecture, word for word, in a separate notebook. Their notes are perfect. Any question you ask, they can flip back and find the exact sentence. The problem is that by week ten, they need a wheelbarrow to carry their notebooks to class.

This is a reasonable description of what a standard Transformer does with its KV cache. For every token in the sequence, the model stores a key vector and a value vector. Ask it to process a document with n tokens and it holds n pairs in memory — growing linearly, with no upper bound. At 100k-token contexts, the cache can consume more GPU RAM than the model weights themselves.

This is not a bug. It is the direct consequence of the exact retrieval guarantee that makes softmax attention so accurate. The question is whether we actually need that guarantee all the time — or whether a student who remembers the gist perfectly is good enough for most exams.

Collapsing the filing cabinet into a single page

Linear attention's insight is deceptively simple: the reason the KV cache grows is that softmax couples every query to every key. If we replace the softmax kernel with a linear one, the sum over all past tokens can be computed incrementally — and the result is a fixed-size state matrix.

Each new token writes to this matrix via an outer product:

$$S \leftarrow S + k(x) v$$

and any query retrieves from it with a simple dot product:

$$o = (S * q) / (z * q), \quad z \leftarrow z + k$$

The state matrix S has shape $d \times d$. Its memory cost is $O(d^2)$ — constant, no matter how many tokens have been processed. The wheelbarrow is gone. In its place is a single dense page of notes.

The write operation is Hebbian: each token adds its key-value contribution to the shared matrix. The read operation is associative: a query key selects a weighted blend of everything written so far. Neuroscientists will recognize this as a content-addressable memory — the same principle that underlies Hopfield networks and, according to Pathway's interpretation, how biological synapses store episodic associations.

"The state is not a cache. It is a compression — and like all compressions, it introduces a loss."

What the page cannot hold

The loss is interference. If two tokens share similar key vectors — say, the tokens "orange" and "yellow" in a sequence — their outer-product writes overlap in the state matrix. A later query for "orange" retrieves a blend: part orange, part yellow, with weights proportional to their key similarity. The student's single-page notes have conflated two different lecture points.

Softmax attention avoids this because the exponential function amplifies differences between scores. Two keys that are 95% similar get scores with a much larger relative gap after exponentiation. The highest-scoring key wins overwhelmingly. Linear attention has no such sharpening — it reports exact proportions, so similar keys genuinely interfere.

State capacity is the second limit. A $d \times d$ matrix can store at most d^2 independent pieces of information. Write more than that — with enough distinct, orthogonal tokens — and earlier writes are progressively overwritten. In practice, real sequences are far from maximally orthogonal, so the effective capacity is higher, but it remains bounded.

Dragon Hatchling: when the state becomes the architecture

Pathway's Dragon Hatchling (BDH) does not treat the recurrent state as an optimization of a Transformer. It treats the state as the primary computational substrate — the thing the model IS, rather than a side effect of how it runs.

Two design decisions address the interference problem directly. First, sparse non-negative activations: roughly 5% of neurons fire on any given token. This reduces write density dramatically — most of the state matrix stays untouched for most tokens, which means collisions between similar keys are much rarer. Second, monosemantic synapses: connections in BDH reportedly encode single semantic concepts rather than the superposed, distributed representations typical of Transformer MLP layers. This further separates the storage regions for different concepts.

BDH-CQ pushes this further: it performs in-context learning — adapting its behavior based on examples provided at inference time — entirely through the recurrent state, with no gradient update. The state is the adapter. This is a meaningful departure from standard in-context learning, where the model's parameters never change but the KV cache carries the demonstration examples. In BDH-CQ, the demonstrations are compressed into state and consumed; the inference loop sees only the final query.

What remains open

Linear attention's core trade-off — constant memory against approximate retrieval — is well characterized. BDH's architectural choices (sparsity, monosemanticity) are theoretically motivated, but independent reproduction of the full system at the reported scales (1B to 600B parameters) is not yet publicly available. Cross-session consolidation — moving fast, recurrent state into slow, durable weights — remains an unsolved problem for all state-space and linear attention architectures. And the right balance between state size and sequence length for production deployments is still an active empirical question.

None of this diminishes the core insight: for many tasks, the student with one well-organized page of notes outperforms the student dragging a wheelbarrow — not because they remember more, but because they show up.

REFERENCES

- [1] Katharopoulos et al. (2020). Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention. ICML 2020. arXiv:2006.16236.
- [2] Pathway (2024). Dragon Hatchling (BDH): A Brain-Inspired Post-Transformer Architecture. Pathway Technical Report.
- [3] Pathway (2024). BDH-CQ: In-Context Learning via Recurrent State. Pathway Technical Report.
- [4] Dao & Gu (2024). Transformers are SSMS: Generalized Models with Structured State Spaces. NeurIPS 2024. arXiv:2405.21060.
- [5] Schlag et al. (2021). Linear Transformers are Secretly Fast Weight Programmers. ICML 2021. arXiv:2102.11174.
- [6] Sun et al. (2023). Retentive Network: A Successor to Transformer for LLMs. arXiv:2307.08621.